Onboarding — Elpis EdgeConnect

Welcome. This document is your single starting point. Read sections 1–4 in order on Day 1. Sections 5–8 are role-specific deep dives; jump to the one that applies to you.

Role	Read these sections in order
Developer (backend / wizard / general)	1, 2, 3, 4, 5, 7, 8
QA engineer	1, 2, 3, 6, 7, 8

This doc points at the existing documentation rather than restating it — your job in week one is to *read what's already written*, not absorb a 50-page summary of it.

---

1. What this project is (one minute)

Elpis EdgeConnect is a protocol-agnostic **Industrial Edge Integration Platform**. It runs as a Windows service (Linux later) on the factory floor, collects data from industrial devices via multiple southbound protocols (FOCAS2, MT-LINKi, MTConnect, Brother HTTP, Modbus TCP, S7, more coming), normalises it through a canonical data pipeline, and delivers it to one or more northbound systems (MQTT, HTTP, TCP, OPC UA Server, more coming).

It is **not** "a gateway with some protocols added over time." It is a modular platform with:

- A **locked Core runtime** (`src/ElpisEdgeConnect.Core/`) that knows nothing about specific protocols
- **Pluggable source and sink adapters** as separate assemblies (`src/ElpisEdgeConnect.Sources.*`, `src/ElpisEdgeConnect.Sinks.*`)
- A **canonical internal data model** (everything becomes `CanonicalDataPoint` before routing)
- **Route-based configuration** (one source can fan out to many sinks; independent per sink)
- **Three-layer licensing** (packaging + signed-license runtime activation + UI/API enforcement)
- A **management UI** ("Connectivity Studio") built in Blazor Server + MudBlazor

Target deployment: factory edge / on-prem. Fully offline-capable. No phone-home licensing.

Sister product: EREMOS V2 (separate project at `C:\dev\EREMOS_V2`) is a downstream MQTT consumer. Cross-project knowledge lives at `C:\dev\shared-knowledge\`.

---

2. First-day setup

2.1 Install prerequisites

Tool	Version	Purpose
.NET SDK	8.0	Build runtime
Git	Any recent	Source control
Visual Studio 2022 / VS Code / Rider	Any	IDE

Python	3.12	Modbus simulator
Mosquitto	Latest	MQTT integration tests
UaExpert	1.7+	OPC UA client (testing)
Browser	Chrome / Edge	Studio UI

`dotnet --list-sdks` should show 8.0.x. If empty, install from https://dotnet.microsoft.com/download.

2.2 Clone + first build

```
cd C:\dev git clone <repo-url> EdgeConnect cd EdgeConnect dotnet build ElpisEdgeConnect.sln
```

Expected: **0 warnings, 0 errors.** If you see warnings, something is wrong with your local environment — flag it before proceeding.

2.3 Run the test suite

```
dotnet test ElpisEdgeConnect.sln --nologo
```

Expected: **~2,500+ tests pass**, 1 skipped (the `Gate5_BrokerOutageReconnect` flaky test). Counts will grow as features ship.

2.4 Run Studio locally

```
cd src\ElpisEdgeConnect.Management\bin\Debug\net8.0 .\ElpisEdgeConnect.Management.exe
```

Studio opens at **`http://127.0.0.1:5080`** (not 5000 — this is the locked binding).

2.5 Verify your shared-knowledge link works

```
ls C:\dev\shared-knowledge\
```

Should show `README.md`, `architecture-overview.md`, `common-modules.md`, `glossary.md`, `contracts/`, `decisions/`. If missing, ask for access to the shared-knowledge repo and clone it. Some of our locked decisions live there.

---

3. Repository tour

```
C:\dev\EdgeConnect\
+-- ElpisEdgeConnect.sln Solution file
+-- CLAUDE.md Project context (read this if you're working with Claude Code)
+-- REVIEW.md Code review checklist
+-- .editorconfig Style + nullable-as-error enforcement
|
+-- docs/
| +-- ARCHITECTURE_BLUEPRINT.md ← Master architecture reference (LOCKED — read this)
| +-- PHASE1_EXECUTION_PLAN.md ← Phase 1 execution plan (history)
| +-- platform-principles.md ← Six cross-milestone principles (LOCKED)
| +-- decisions/ ← ADRs — short locked decisions, numbered (0001..NNNN)
| +-- sessions/ ← Per-session plans and handoffs (the plan trail)
```

```
| +-- core/ Core runtime internal docs
| +-- adapter-sdk/ Adapter SDK guides
| +-- benchmarks/ Captured benchmark baselines
| +-- config-schemas/ Generated JSON schemas
| +-- licensing/ License format spec + module catalog
| +-- qa/ QA plans + trackers
| \-- onboarding.md ← This file
|
+-- src/
| +-- ElpisEdgeConnect.Core/ Protocol-agnostic core runtime
| +-- ElpisEdgeConnect.Management/ Connectivity Studio (Blazor Server)
| +-- ElpisEdgeConnect.Host/ Windows-service / console host
| +-- ElpisEdgeConnect.Sources.Focas2/ FANUC FOCAS2 source adapter
| +-- ElpisEdgeConnect.Sources.BrotherHttp/ Brother HTTP source adapter
| +-- ElpisEdgeConnect.Sources.ModbusTcp/ Modbus TCP source adapter
| +-- ElpisEdgeConnect.Sources.MTConnect/ MTConnect source adapter
| +-- ElpisEdgeConnect.Sources.S7/ Siemens S7 source adapter
| +-- ElpisEdgeConnect.Sinks.Mqtt/ MQTT sink adapter
| +-- ElpisEdgeConnect.Sinks.OpcUaServer/ OPC UA Server sink adapter
| +-- ElpisEdgeConnect.MockAdapters/ Reference / mock adapters for tests
| \-- ElpisEdgeConnect/ LEGACY — original FanucCncDataBridge code. Don't add new code here.
|
+-- tests/
| +-- ElpisEdgeConnect.Core.Tests/ xUnit + FluentAssertions + NSubstitute
| +-- ElpisEdgeConnect.Management.Tests/ Mgmt-layer tests; bUnit for Razor components
| +-- ElpisEdgeConnect.Host.Tests/
| +-- ElpisEdgeConnect.Sources.*.Tests/ One test project per protocol
| +-- ElpisEdgeConnect.Sinks.*.Tests/
| +-- ElpisEdgeConnect.Integration.Tests/ End-to-end (needs Docker, Mosquitto, etc.)
| \-- ElpisEdgeConnect.Benchmarks/ BenchmarkDotNet
|
\-- tools/ CLI utilities (LicenseGen, SchemaGen, ModbusSoakRunner, etc.)
```

Dependency direction is one-way: `Core ← Adapters ← Management ← Host`. Never the other way around. Core knows nothing about protocols.

---

4. The documents that govern everything

Read these in order. Anything that contradicts them is wrong by definition.

Document	Purpose	Read by
`docs/ARCHITECTURE_BLUEPRINT.md`	Master architecture reference. 19 sections + Appendix A listing LOCKED / FLEXIBLE / OPEN decisions.	Everyone — Day 1
`docs/ platform-principles.md`	Six cross-milestone commitments shaping every decision (P1 Runtime Tap observational; P2 shared primitives; P3 security spec-first; P4 explainability; P5 EREMOS V2 market identity; P6 operational product).	Everyone — Day 1

`docs/ decisions/ 0001..NNNN.md`	ADRs — locked decisions, one file each. Read ADR 0001, 0002, 0008, 0014, 0015 as the most load-bearing.	Everyone — Week 1
`docs/ PHASE1_EXECUTION_PLAN.md`	Phase 1 execution plan (historical — Phase 1 closed).	Devs — for context
`CLAUDE.md` (repo root)	Working context for Claude Code sessions. Useful even if you're not using Claude — captures locked conventions in one place.	Everyone — Day 1
Latest session in `docs/ sessions/`	Per-session plans and handoffs. **Always read the most recent file before starting work** — it tells you what just shipped and what's in flight.	Everyone — every session

Critical convention: If a decision you're about to make isn't covered by the blueprint, the platform principles, or an ADR — **stop and surface it to the user** rather than choosing silently. We have a culture of "pause and report rather than silently simplify."

---

5. For developers — building a feature end-to-end

5.1 The development loop

Step	Activity	Lives in
1	Read the latest session handoff	`docs/ sessions/ <latest>.md`
2	Write a v1 plan (brief, in your own file)	`docs/ sessions/ <date>-<milestone>-plan.md`
3	Run the plan past the user / ChatGPT for review	(offline)
4	Lock decisions → write v2 plan, or v2.1 if minor refinements	`docs/ sessions/ <date>-<milestone>-plan-v2.md`
5	Write or update the relevant ADR **before code** if you're locking a contract	`docs/ decisions/ NNNN-<topic>.md`
6	Implement on a branch (`claude/ <milestone>-impl` or similar)	source folders
7	TDD where it makes sense (tests first, then implementation)	matching `tests/` folder
8	Full test sweep, zero warnings, no new flaky tests	`dotnet test ElpisEdgeConnect.sln`
9	Commit with descriptive 1–2 sentence message focusing on *why*	git

10	Push + open PR via `gh pr create`	GitHub
11	Write a handoff doc	`docs/ sessions/ <date>-<milestone>-handoff.md`

5.2 Coding conventions (locked)

- **Namespaces match folder structure** — `ElpisEdgeConnect.Core.Model`, `ElpisEdgeConnect.Core.Adapters`, etc.
- **Private fields** use `_camelCase` prefix.
- **Records** are `sealed` unless there's an explicit reason to allow inheritance.
- **`required` init properties** for mandatory fields, not constructor parameters.
- **No `async void`** except event handlers.
- **Library async calls** use `ConfigureAwait(false)` where applicable.
- **Error codes** follow `MODULE.CATEGORY_SUBCATEGORY` — `CORE.CONFIG_INVALID`, `FOCAS2.HANDLE_EXHAUSTED`, `MODBUS.CONNECT_TIMEOUT`.
- **Every source file** starts with a header comment naming the file, purpose, and blueprint section it implements.
- **Every public member** in Core has XML doc comments.
- **`TreatWarningsAsErrors=true`** on `ElpisEdgeConnect.Core` — no exceptions.
- **Cross-platform:** code must build and run on Linux. Avoid Windows-only APIs except behind `OperatingSystem.IsWindows()` guards.

5.3 Testing conventions

- **Test class:** `{ClassUnderTest}Tests`
- **Test method:** `MethodName_Condition_ExpectedResult`
- **Arrange–Act–Assert** with blank lines separating phases
- **Every locked architectural requirement** must have at least one named test that fails if violated
- **No `Thread.Sleep`** in tests — use `TaskCompletionSource` or time abstractions for async signals
- **bUnit** is the established pattern for Razor component tests in `ElpisEdgeConnect.Management.Tests` (we use the loose JSRuntime mode for non-interop tests, strict mode for tests that assert on JS calls)
- **Code coverage on Core** must be ≥80%
- **Integration tests** that need Docker / Mosquitto skip gracefully if those aren't available

5.4 The wizard contract (if you're touching Studio)

If you're adding a new protocol's wizard, follow **ADR-0015** (`docs/decisions/0015-wizard-contract.md`). The 5-step operational guide at the bottom of that ADR is the answer to "how do I add a new wizard." Don't read the existing six wizards and reverse-engineer the pattern — read the ADR.

5.5 Adding a new adapter

If you're adding a new protocol (source or sink):

1. Read `docs/adapter-sdk/` — the SDK guides are the contract.
2. Create `src/ElpisEdgeConnect.Sources.<Protocol>/` (or `Sinks.`).

3. Implement `ISourceAdapter` / `ISinkAdapter` from `ElpisEdgeConnect.Core.Adapters`.
4. Write per-instance items as separate validatable types if applicable (`MyDataPoint`, `MyTagDefinition`, etc.).
5. Per **ADR-0015 Rule 2**, ship a `static class FooValidator` with `Validate(item, pathPrefix, errors)` if you have per-item validation. The adapter's `ValidateConfigAsync` and the wizard model both call it.
6. Add a test project `tests/ElpisEdgeConnect.Sources.<Protocol>.Tests/`.
7. Register the protocol in the host's DI wire-up (license-gated).
8. Add a wizard if there's user-facing configuration.

5.6 Common gotchas

Gotcha	Symptom	Solution
Studio running locks `Management.dll`	`MSB3027: file locked` during rebuild	Stop Studio (Ctrl+C in its terminal) before rebuilding
MQTT integration tests fail	`BrokerOffline` or connection refused	Start Mosquitto on `localhost:1883` (anonymous)
Tests slow on first run	Lots of `dotnet restore` activity	First-run only; subsequent runs faster
Razor component changes not visible	Old DLL still loaded	Restart Studio after changing `.razor` files (Blazor hot reload doesn't always pick up structural changes)
`WizardValidationBanner` empty when expected	Validation list returns null/empty	Banner intentionally renders zero DOM in this case — ADR-0015 Rule 5. Absence is the success signal.
Edit wizard loading spinner forever	`_currentConfig` null in edit mode	Edit mode skips loading config; check the spinner guard is `_currentConfig is null && !_isEdit`

5.7 The plan trail

We use a versioned planning convention:

- **v1** — a brief draft, often with open questions
- **review** — ChatGPT or peer pass
- **v2** — open questions locked, ratified by user
- **reality-check** — implementation pass surfaces gaps
- **v3** / **v2.1** — refinement after reality-check

Each version lives in its own dated file under `docs/sessions/`. **Don't overwrite the previous version** — the trail is the audit log of how decisions evolved.

---

6. For QA — running the test plan

6.1 Start here

Resource	What it is
`docs/ qa/ 2026-05-27-modbus-to-opcua-pipeline-qa-plan.md`	The full Modbus → OPC UA pipeline test plan — 67 test cases, ~16 pages
`docs/ qa/ 2026-05-27-modbus-to-opcua-pipeline-qa-tracker.xlsx`	Live tracking spreadsheet — fill Result column per case, Cover sheet stats auto-update
`tests/ ElpisEdgeConnect.Integration.Tests/ ModbusSimulator/ README.md`	How to run the pymodbus-based Modbus simulator (your primary test fixture)
`tests/ ElpisEdgeConnect.Management.Tests/ Wizards/ CrossWizardConsistencyAuditChecklist.md`	Cross-wizard consistency audit (locked at M.2d.4)

6.2 Daily QA workflow

1. **Morning:** read the latest session handoff in `docs/sessions/` to know what shipped overnight.
2. **Pull latest:** `git pull` on `master`, then check out the branch under test (usually `claude/<milestone>-impl`).
3. **Build + run:** `dotnet build` then start Studio.
4. **Execute test cases** from the QA tracker spreadsheet. Filter by Priority = P1 first (smoke gate). Move to P2/P3 once smoke is green.
5. **For every Fail:** fill Severity (S1..S5) + Defect ID (link to bug tracker) + Notes. Severity rubric is in the QA plan §2.6.
6. **End-of-day:** push the updated tracker to a shared location (commit to git if that's the convention, or upload to drive).

6.3 QA-relevant infrastructure

What	Where	Setup time
Modbus simulator	`tests/ ElpisEdgeConnect.Integration.Tests/ ModbusSimulator/` (Python, pymodbus<3.8)	5 min first time, instant after
Mosquitto MQTT broker	https://mosquitto.org/ download/ — install + run on `localhost:1883` (anonymous)	5 min
UaExpert OPC UA client	https://www.unified-automation.com/ products/ development-tools/ uaexpert.html	10 min including registration
Wireshark	https://www.wireshark.org/ — for raw protocol inspection if needed	5 min
`mosquitto_sub` CLI	Ships with Mosquitto. Use for headless MQTT subscription testing.	—

6.4 Defect template

Use the template in QA plan §15. Critical fields:

```
TC ID: TC-?-???
Severity: S1 / S2 / S3 / S4 / S5
Title: <one-line summary>
Steps to reproduce: 1. ... 2. ...
Expected: ...
Actual: ...
Environment: EdgeConnect commit <SHA> / Windows 11 / pymodbus <ver> / UaExpert <ver>
Logs: <attach>
```

6.5 What QA should NOT be expected to find

Documented out-of-scope in QA plan §14:

- Authentication beyond Basic auth (OAuth/OIDC is future work)
- OPC UA Sign / SignAndEncrypt enforcement (configurable but not yet runtime-enforced — Milestone K)
- HA / clustering (single-instance only)
- Performance beyond ~100 tags (capacity testing is a separate plan)

If you stumble on something matching this list and it doesn't work, document it as a "behavioural confirmation" not a defect.

6.6 Communication

- **Daily defect triage** — quick sync between dev + QA to classify new defects and prioritise fixes.
- **Defects need reproducible steps.** "It crashed once" is not a defect; "It crashed at TC-R-001 step 4 after killing the simulator three times in 60 seconds" is.
- **When in doubt, file the defect.** Better a duplicate than a missed bug.

---

7. Anti-patterns to refuse

These are from the project's CLAUDE.md and apply to everyone. Refuse or push back if asked to do any of these without explicit user override:

1. **Adding protocol-specific logic to `ElpisEdgeConnect.Core`.** Core is protocol-agnostic. Period.
2. **Loading assemblies dynamically at runtime** to simulate plugin loading. v1 uses compile-time projects with license-gated DI registration.
3. **Putting AI agents in the data path.** Agents live in the management layer. The pipeline stays deterministic.
4. **Implementing `ExactlyOnce` delivery mode.** Out of scope for v1 per blueprint §19.7.
5. **Transactional fanout across sinks.** Sinks commit independently per blueprint §19.2.
6. **Global ordering guarantees across sources.** Only per-source ordering is promised.
7. **Requiring cloud LLM access for AI features.** Local-LLM support is mandatory from day one.
8. **Phoning home for license validation.** Licenses are fully offline.
9. **Silent AI actions that change state.** All state changes require explicit user confirmation.
10. **Skipping the draft → validate → apply → rollback flow for config changes.** Even in tests, the flow must be honoured.
11. **Referencing a protocol module from another protocol module.** Dependency direction is strictly Core ← Adapters.

12. **Adding a new error code without putting it in `CoreErrors.cs`** (or the equivalent catalog for protocol modules).
13. **Auto-saving wizard state to `localStorage`.** ADR-0015 Rule 8 forbids this.

If a user request seems to conflict with these, **surface the conflict before proceeding.**

---

8. Current state of play

8.1 Phase status

Phase	Status
Phase 1 — Core foundation	**Closed** (tag `v0.1.0-phase1`) — canonical model, contracts, pipeline, routing, buffer, licensing, diagnostics
Phase 2 — Protocol adapters	**In progress** — FOCAS2, Brother HTTP, Modbus, MQTT, OPC UA Server all shipped; MT-LINKi pending
Phase 3 — Edition installers + license-gated activation	Not started
Phase 4 — Management Studio	**In progress** — M.2b (Connectivity Studio core), M.2c (Runtime Tap, not started), M.2d (Wizard polish, in progress)
Phase 4.5 — AI agents	Not started
Phase 5 — Advanced (OPC UA Client, S7, fleet management)	Not started

8.2 Most recent work

Always check `docs/sessions/` for the latest. As of this onboarding doc (2026-05-27):

- **M.2d.4** — Cross-wizard consistency sweep is the current active milestone. Branch `claude/m2d4-impl`. Goal: all five protocol wizards conform to ADR-0015's wizard contract.
- **PR #37 (M.2d.3)** — merged to master: sink + route edit-via-wizard with optimistic concurrency.
- **ADR-0015** — newly landed: the wizard contract.

8.3 First-month reading list (sequential)

Week 1: 1. `docs/onboarding.md` — this file 2. `CLAUDE.md` 3. `docs/ARCHITECTURE_BLUEPRINT.md` (read fully — it's the master reference) 4. `docs/platform-principles.md` 5. `docs/decisions/0001-canonical-data-model.md` through `docs/decisions/0014-config-state-vs-runtime-state.md` (skim; bookmark for re-reading) 6. The most recent 5 files in `docs/sessions/`

Week 2: 7. `docs/decisions/0015-wizard-contract.md` 8. `src/ElpisEdgeConnect.Core/` — at least walk the folder structure and read one file per subfolder 9. `tests/ElpisEdgeConnect.Core.Tests/` — read 3–5 representative test files 10. `docs/PHASE1_EXECUTION_PLAN.md` (historical context — Phase 1 is closed, but the patterns there are the patterns we still use)

Week 3 (developer-specific): 11. Pick one adapter (recommend `src/ElpisEdgeConnect.Sources.ModbusTcp/`) and read it end-to-end including its tests 12. Pick one wizard (recommend

`src/ElpisEdgeConnect.Management/Components/Pages/SourceWizards/AddBrotherHttpSource.razor`) and read it including the underlying `*WizardModel.cs` 13. The current active milestone's plan v2 (whatever that is) — read enough to understand what's in flight

Week 3 (QA-specific): 11. `docs/qa/2026-05-27-modbus-to-opcua-pipeline-qa-plan.md` — read fully 12. Execute the QA plan's P1 cases end-to-end in your local environment as a learning exercise (you'll file your first defect this week if all goes well) 13. `tests/ElpisEdgeConnect.Integration.Tests/` — at least open the README and one test class to see how the framework expects fixtures to be set up

Week 4: Pick up a real piece of work. For developers, that's a small follow-up chip (see `docs/sessions/*-followup-chips.md`). For QA, that's a full milestone test run with defect filing.

---

9. Glossary

Term	Meaning
ADR	Architecture Decision Record. Short, numbered, locked-once-accepted. Lives in `docs/ decisions/`.
Adapter	A protocol-specific module that translates between an external protocol and the canonical pipeline. Source adapters poll/subscribe to devices; sink adapters publish to external systems.
Apply	The step that promotes a draft into the running configuration. Triggers adapter restarts where needed.
Brother HTTP	A CNC controller HTTP polling protocol (proprietary to Brother machines).
Canonical Data Point	The internal normalised representation of a single tag value flowing through EdgeConnect. Protocol-agnostic.
Connectivity Studio	The Blazor Server admin UI. Lives in `src/ ElpisEdgeConnect.Management/`. Binds at `127.0.0.1:5080`.
Destination	Operator-facing name for "Sink" (per ADR-0008 — UI says destinations, code says sinks).
Draft	A pending configuration change that hasn't been Validate+Apply'd yet. Persisted server-side.
EREMOS V2	Downstream MQTT consumer / cloud product. Separate project but shares MQTT contract.
FOCAS2	A FANUC CNC protocol (Focas Library). Source adapter at `src/ ElpisEdgeConnect.Sources.Focas2/`.
Modbus TCP	Industrial polling protocol. Source adapter polls registers/coils at a configured rate.
MT-LINKi	A FANUC fleet-management protocol layered on FOCAS2.
MTConnect	An open machine-tool data protocol (HTTP/XML).

Route	A wiring object — pairs one source with one or more sinks, with optional filter + transform pipeline.
Sink	A destination — receives canonical data and pushes to MQTT, OPC UA Server, HTTP, etc. UI calls these "Destinations".
Source	A protocol adapter that polls or subscribes to an external device.
Store-and-forward (SAF)	The per-route SQLite buffer that holds messages while downstream sinks are unreachable.
UaExpert	A widely-used OPC UA client. Our primary test client for the OPC UA Server sink.
WizardShell / WizardSection / WizardValidationBanner / WizardActions	Shared Blazor primitives every protocol wizard composes from. Locked in ADR-0015.

---

10. Who to ask

- **Architecture questions** that aren't answered in the blueprint or ADRs → escalate before guessing.
- **Pipeline data issues** → check `docs/decisions/0014-config-state-vs-runtime-state.md` first; the question is often "is this a config issue or a runtime issue?"
- **Wizard / Studio questions** → ADR-0015 is the contract; sessions/2026-05-2-*m2d* are the implementation history.
- **License / packaging questions** → `docs/licensing/`.
- **Anything in flight** → the latest session handoff in `docs/sessions/`.

When you're stuck, the order is: (1) re-read the relevant section of the blueprint, (2) re-read the relevant ADR, (3) re-read the most recent session, (4) ask. The worst outcome is silently making an architectural choice that later has to be unwound. Asking a question that's already answered in the docs is the second-worst, but it's much better than the first.

Welcome aboard.